

Capacité numérique 2 : Détermination de l'état final d'un système physico-chimique

PTSI2 La Martinière Monplaisir

Année 2024-2025

But : déterminer, à l'aide d'un langage de programmation, l'état final d'un système, siège d'une transformation, modélisée par une réaction à partir des conditions initiales et valeur de la constante d'équilibre.

On montre ici comment résoudre l'équation pour l'avancement obtenue à partir de l'écriture de la constante d'équilibre.

1 Position du problème

On considère une réaction décrite par une équation-bilan sous la forme

$$\sum_k \nu_k A_k = 0.$$

On connaît les quantités initiales des différentes espèces A_k , notées $n_{k,0}$, ainsi que la valeur de la constante d'équilibre, notée $K^\circ(T)$. La température est supposée fixée.

À l'équilibre, la loi d'action de masse donne une relation entre les activités a_k des espèces A_k :

$$K^\circ(T) = \prod_k a_k^{\nu_k}.$$

Connaissant l'expression de l'activité, on cherche à déterminer la valeur de l'avancement à l'équilibre. On pourra ainsi en déduire la composition du système à l'équilibre.

Au vu des différentes expressions possibles de l'activité, il n'est pas possible de donner de résultat général. On doit procéder au cas par cas.

2 Exemple d'une réaction d'oxydoréduction

On considère la réaction d'oxydoréduction suivante, entre les couples IO_3^- / HIO et $Br_2(aq) / Br^-$:



dont la constante d'équilibre à $T = 298\text{ K}$ vaut $K^\circ = 5.15$. La réaction n'est ni totale ni défavorisée, on ne peut donc pas faire d'approximation simple pour calculer l'avancement à l'équilibre.

Dans un volume $V = 500\text{ mL}$, on prendra les quantités de matière initiales suivantes :

$$\begin{aligned} n_{IO_3^-,0} &= 0.1\text{ mol} & n_{H^+,0} &= 0.5\text{ mol} & n_{Br^-,0} &= 0.2\text{ mol} & n_{HIO,0} &= 0.1\text{ mol} \\ n_{Br_2,0} &= 0.2\text{ mol}. \end{aligned}$$

2.1 Tableau d'avancement

Afin de déterminer l'équation qu'on cherchera à résoudre numériquement, on effectue d'abord un tableau d'avancement :

	1IO_3^-	+	5H^+	+	4Br^-	=	$1\text{HIO}(aq)$	+	$2\text{H}_2\text{O}$	+	2Br_2
EI	$n_{\text{IO}_3^-,0}$		$n_{\text{H}^+,0}$		$n_{\text{Br}^-,0}$		$n_{\text{HIO},0}$		excès		$n_{\text{Br}_2,0}$
EF	$n_{\text{IO}_3^-,0} - 1\xi$		$n_{\text{H}^+,0} - 5\xi$		$n_{\text{Br}^-,0} - 4\xi$		$n_{\text{HIO},0} + 1\xi$		excès		$n_{\text{Br}_2,0} + 2\xi$

EI et EF sont les abréviations d'État Initial et État Final, ce dernier correspondant à l'état d'équilibre. On peut éventuellement rajouter une ligne correspondant à un état intermédiaire entre EI et EF.

Remarque : On a ici établi un tableau d'avancement en quantité de matière, ce qui est préférable en général. Dans le cas où le volume ne varie pas (comme ici), on peut alors passer à un tableau d'avancement en concentration. Mais de manière générale, et c'est tout particulièrement vrai pour les dosages et les réactions dans lesquelles interviennent des espèces sous plusieurs phases, il est préférable de conserver les quantités de matière dans le tableau d'avancement.

En termes des concentrations et de l'avancement volumique $x = \xi/V$, le tableau d'avancement devient

	IO_3^-	+	5H^+	+	4Br^-	=	$\text{HIO}(aq)$	+	$2\text{H}_2\text{O}$	+	2Br_2
EI	$[\text{IO}_3^-]_0$		$[\text{H}^+]_0$		$[\text{Br}^-]_0$		$[\text{HIO}]_0$		excès		$[\text{Br}_2]_0$
EF	$[\text{IO}_3^-]_0 - x$		$[\text{H}^+]_0 - 5x$		$[\text{Br}^-]_0 - 4x$		$[\text{HIO}]_0 + x$		excès		$[\text{Br}_2]_0 + 2x$

2.2 Écriture de la constante d'équilibre - Équation pour l'avancement

Chacune des espèces intervenant dans la réaction est soluble, ce qui permet d'écrire l'activité de l'espèce i sous la forme $a_i = \frac{C_i}{C^\circ}$.

On utilise la définition de la constante d'équilibre, pour trouver

$$K^\circ = \frac{(C^\circ)^7 ([\text{Br}_2]_0 + 2x)^2 \cdot ([\text{HIO}]_0 + x)}{([\text{IO}_3^-]_0 - x) \cdot ([\text{H}^+]_0 - 5x)^5 \cdot ([\text{Br}^-]_0 - 4x)^4}.$$

On en déduit l'équation vérifiée par l'avancement volumique x :

$$(C^\circ)^7 ([\text{Br}_2]_0 + 2x)^2 \cdot ([\text{HIO}]_0 + x) - K^\circ \cdot ([\text{IO}_3^-]_0 - x) \cdot ([\text{H}^+]_0 - 5x)^5 \cdot ([\text{Br}^-]_0 - 4x)^4 = 0.$$

Cette équation ne peut pas être résolue exactement, il faut utiliser un outil numérique.

3 Résolution numérique

3.1 Méthode par dichotomie

3.1.1 Principe

On considère une fonction $f(x)$ continue sur un intervalle $[a, b]$. Avec $f(a)$ et $f(b)$ de signe opposés ce qui se traduit par : $f(a) \cdot f(b) < 0$. Le théorème des valeurs intermédiaires nous assure alors que f possède au moins un zéro (noté ℓ) entre a et b . Posons $g_0 = a$ et $d_0 = b$.

Initialisation de la procédure

On considère : $m_0 = \frac{g_0 + d_0}{2}$ et on évalue $f(m_0)$:

— Si $f(m_0) \cdot f(a) \leq 0$, on va poursuivre la recherche d'un zéro dans l'intervalle $[g_0, m_0]$

On pose donc : $g_1 = g_0$ et $d_1 = m_0$.

— Sinon, la recherche se poursuit dans l'intervalle $[m_0, d_1]$.

On pose donc : $g_1 = m_0$ et $d_1 = d_0$

Récurrence

On a défini à l'étape précédente les valeurs de d_n et g_n . On calcule $m_n = \frac{g_n + d_n}{2}$.

- Si $f(m_n) \cdot f(g_n) \leq 0$, on va poursuivre la recherche d'un zéro dans l'intervalle $[g_n, m_n]$
On pose donc : $g_{n+1} = g_n$ et $d_{n+1} = m_n$.
- Sinon, la recherche se poursuit dans l'intervalle $[m_n, d_n]$.
On pose donc : $g_{n+1} = m_n$ et $d_{n+1} = d_n$

Condition d'arrêt

On s'arrête lorsque $d_n - g_n < 2\varepsilon$, avec ε la précision souhaitée. on posera alors

$$\ell = m_n = \frac{d_n + g_n}{2}.$$

3.1.2 Script pour la dichotomie

```

1 import numpy as np # Outils numeriques
2 import matplotlib.pyplot as plt # Outils graphiques
3 from scipy.optimize import fsolve
4
5 #Concentrations initiales
6 #Des reactifs
7 CI03_0 = 0.1
8 CH_0 = 0.5
9 CBr_0 = 0.2
10 #Des produits
11 CBr2_0 = 0.2
12 CHIO_0 = 0.1
13
14 #Concentration standard et constante d equilibre
15 C0 = 1. #C0 = 1 mol/L
16 K0 = 5.15
17
18 #Definition de la fonction correspondant a la condition d equilibre
19 def func(x):
20     return (C0)**7*(CBr2_0+2*x)**2*(CHIO_0+x)-K0*(CI03_0-x)*(CH_0-5*x)**5*(CBr_0
21         -4*x)**4
22
23 #Quotient de reaction pour verifier le sens d'evolution
24 def Qr(x):
25     return ((C0)**7*(CBr2_0+2*x)**2*(CHIO_0+x))/((CI03_0-x)*(CH_0-5*x)**5*(CBr_0
26         -4*x)**4)
27
28 print('Qr initial =', Qr(0))
29
30 if(Qr(0)>K0):
31     print('Qr(initial)>K0 => Evolution dans le sens <-')
32     a,b=-1,0
33 else:
34     print('Qr(initial)<K0 => Evolution dans le sens ->')
35     a,b=0,1
36
37 ##Fonction pour la dichotomie
38 def dichotomie (f, a, b, epsilon):
39     g=a
40     d=b
41     while d-g>2*epsilon:
42         m = (g+d)/2
43         if f(g)*f(m)<=0:
44             d=m
45         else:
46             g=m
47     return ((g+d)/2)
48
49 ##Calcul de l avancement a l equilibre
50 xeq = dichotomie(func, -1., 0., 0.001)

```

```

50 # xeq = fsolve(func, 0)[0]
51
52 #Affichage des resultats, et calcul des concentrations a l'equilibre
53 #La fonction round(nombre,N) arrondit nombre a la Nieme decimale
54 print("Avancement volumique a l'equilibre = ",round(xeq,3),"mol/L")
55 print("Concentrations a l'equilibre:\n")
56 print('[I03-] = ', round(CI03_0-xeq,3),"mol/L")
57 print('[Br2] = ', round(CBr2_0+2*xeq,3),"mol/L")
58 print('[HIO] = ', round(CHIO_0+xeq,3),"mol/L")
59 print('[H+] = ', round(CH_0-5*xeq,3),"mol/L")
60 print('[Br-] = ', round(CBr_0-4*xeq,3),"mol/L")

```

3.2 En utilisant une dichotomie et bisect

On peut proposer le code suivant, qui utilise la fonction bisect :

```

1 #Resolution de l equation obtenue avec la condition d equilibre
2 #avec une dichotomie utilisant la fonction fsolve
3
4 import numpy as np # Outils numeriques
5 import matplotlib.pyplot as plt # Outils graphiques
6 from scipy.optimize import fsolve
7
8 #Concentrations initiales
9 #Des reactifs
10 CI03_0 = 0.1
11 CH_0 = 0.5
12 CBr_0 = 0.2
13 #Des produits
14 CBr2_0 = 0.2
15 CHIO_0 = 0.1
16
17 #Concentration standard et constante d'equilibre
18 C0 = 1. #C0 = 1 mol/L
19 K0 = 5.15
20 #Quotient de reaction pour verifier le sens d'evolution
21 def Qr(x):
22     return ((C0)**7*(CBr2_0+2*x)**2*(CHIO_0+x))/((CI03_0-x)*(CH_0-5*x)**5*(CBr_0-4*x)**4)
23
24 print('Qr initial =',Qr(0))
25
26 if(Qr(0)>K0):
27     print('Qr(initial)>K0 => Evolution dans le sens <-')
28 else:
29     print('Qr(initial)<K0 => Evolution dans le sens ->')
30
31
32 #Definition de la fonction correspondant a la condition d'equilibre
33 def func(x):
34     return (C0)**7*(CBr2_0+2*x)**2*(CHIO_0+x)-K0*(CI03_0-x)*(CH_0-5*x)**5*(CBr_0-4*x)**4
35
36 #La commande fsolve(func,0) resoud l'equation func(x)=0
37 #Le resultat est un tableau numpy dont le premier argument est la valeur de xeq
38 xeq = fsolve(func, 0)[0]
39
40 #Affichage des resultats, et calcul des concentrations a l'equilibre
41 #La fonction round(nombre,N) arrondit nombre a la Nieme decimale
42 print("Avancement volumique a l'equilibre = ",round(xeq,3),"mol/L")
43 print("Concentrations a l'equilibre:\n")
44 print('[I03-] = ', round(CI03_0-xeq,3),"mol/L")
45 print('[Br2] = ', round(CBr2_0+2*xeq,3),"mol/L")
46 print('[HIO] = ', round(CHIO_0+xeq,3),"mol/L")
47 print('[H+] = ', round(CH_0-5*xeq,3),"mol/L")
48 print('[Br-] = ', round(CBr_0-4*xeq,3),"mol/L")

```

Avec les conditions initiales suggérées ci-dessus, on obtient les résultats suivants à 298 K :

Avancement volumique à l'équilibre : $x_{eq} = -0.034 \text{ mol/L}$

Concentrations à l'équilibre :

$$[IO_3^-] = 0.134 \text{ mol/L}$$

$$[Br_2] = 0.132 \text{ mol/L}$$

$$[HIO] = 0.066 \text{ mol/L}$$

$$[H^+] = 0.669 \text{ mol/L}$$

$$[Br^-] = 0.335 \text{ mol/L}$$

3.3 Variante avec la fonction fsolve

```
1 #Resolution de l equation obtenue avec la condition d equilibre
2 #avec une dichotomie utilisant la fonction bisect
3
4 import numpy as np # Outils numeriques
5 import matplotlib.pyplot as plt # Outils graphiques
6 from scipy.optimize import bisect
7
8 #Concentrations initiales
9 #Des reactifs
10 CI03_0 = 0.1
11 CH_0 = 0.5
12 CBr_0 = 0.2
13 #Des produits
14 CBr2_0 = 0.2
15 CHIO_0 = 0.1
16
17 #Concentration standard et constante d'equilibre
18 C0 = 1. #C0 = 1 mol/L
19 K0 = 5.15
20
21
22 #Quotient de reaction pour verifier le sens d'evolution
23 def Qr(x):
24     return ((C0)**7*(CBr2_0+2*x)**2*(CHIO_0+x))/((CI03_0-x)*(CH_0-5*x)**5*(CBr_0
25         -4*x)**4)
26
27
28 #Definition de la fonction correspondant a la condition d'equilibre
29 def f(x):
30     return (C0)**7*(CBr2_0+2*x)**2*(CHIO_0+x)-K0*(CI03_0-x)*(CH_0-5*x)**5*(CBr_0
31         -4*x)**4
32
33 #La commande bisect(f,a,b) resoud l equation f(x)=0 sur l intervalle [a,b].
34 #Elle s'assure que f(a)*f(b)<0 (changement de signe)
35 #Si on se trouve dans le cas Q<K0 alors on choisit a = 0 et on prend b
36 #de sorte a verifier f(a)*f(b)<0
37 #Si on se trouve dans le cas Q>K0 alors on choisit b = 0 et on prend a
38 #de sorte a verifier f(a)*f(b)<0
39
40 if(Qr(0)>K0):
41     print('Qr(initial)>K0 => Evolution dans le sens <-')
42     xeq = bisect(f,-0.5,0.)
43 else:
44     print('Qr(initial)<K0 => Evolution dans le sens ->')
45     xeq = bisect(f,0.,0.5)
46
47 #Affichage des resultats, et calcul des concentrations a l'equilibre
48 #La fonction round(nombre,N) arrondit nombre a la Nieme decimale
49 print("Avancement volumique a l'equilibre = ",round(xeq,3),"mol/L")
50 print("Concentrations a l'equilibre:\n")
51 print('[IO3-] = ', round(CI03_0-xeq,3),"mol/L")
```

```
51 print(' [Br2] = ', round(CBr2_0+2*xeq,3), "mol/L")
52 print(' [HIO] = ', round(CHIO_0+xeq,3), "mol/L")
53 print(' [H+] = ', round(CH_0-5*xeq,3), "mol/L")
54 print(' [Br-] = ', round(CBr_0-4*xeq,3), "mol/L")
```

Le résultat est bien sûr identique. La fonction `fsolve` est plus flexible que `bisect`, elle s'exécute automatiquement sans préciser d'intervalle de recherche.